

Dart Fundamentals - Assignment 2 Notes

Topic: Null Safety, Future, async/await, and Error Handling

1. Null Safety in Dart

Null means a variable has no value (it is empty). In older versions of programming languages, trying to use a null variable would crash the entire application at runtime. Dart uses Null Safety to stop these crashes before the code even runs.

- **Non-nullable (Default):** Variables cannot be null. Example: `String name = 'Alice';` (Cannot be set to null).
- **Nullable (?):** Adding '?' means the variable is allowed to be null. Example: `String? email;`
- **Null Coalescing (??):** Used to give a default value if something is null. Example: `email ?? 'Not Provided'`
- **Safe Call (?.):** Only calls a property or method if the object is not null. Example: `user?.name`

Code Example:

```
class User {  
  final int id;          // Required: Cannot be null  
  final String name;     // Required: Cannot be null  
  final String? email;   // Optional: Can be null  
  final int? age;        // Optional: Can be null  
  
  User({required this.id, required this.name, this.email, this.age});  
}
```

2. Asynchronous Programming (Future, async, await)

Normally, code runs line by line (synchronously). If a task takes time (like loading data from the internet), running synchronously freezes the entire program. Asynchronous programming lets time-consuming tasks run in the background so the app stays fast and responsive.

- **Future<T>:** Represents a promise of a value of type T that will arrive in the future.
- **async:** A keyword added to a function to tell Dart that it will run asynchronous tasks.
- **await:** Tells Dart to pause inside that function and wait for the Future to finish before moving to the next line.

Simple Real-Life Analogy:

When you order food at a restaurant counter, you get an order token (Future). You don't freeze and block other customers. When your food is ready (await), you receive your meal.

Code Example:

```
// Simulates downloading user info from an API  
Future<User> fetchUserData(int userId) async {  
  print("Fetching data for User ID: $userId...");  
  await Future.delayed(Duration(seconds: 1)); // Wait 1 second  
  return User(id: userId, name: "Alice", email: "alice@example.com");  
}
```

3. Error Handling (try-catch)

API calls can fail for many reasons: no internet connection, server down, or invalid user ID. We use try-catch blocks to catch errors and prevent crashes.

- **try block:** Put code here that might cause an error or exception.
- **catch (e):** If an error happens inside 'try', code jumps here so you can show a friendly message.
- **throw:** Used to manually trigger an error when something goes wrong (e.g. invalid user ID).

Code Example:

```
try {
  User user = await fetchUserData(-5); // Invalid ID
  displayUser(user);
} catch (e) {
  print("Error caught safely: $e");
}
```

4. Assignment 2 (fetch_data.dart) Cases Explained

- **Case 1 (Success):** User ID 1 is fetched with all fields (name, email, age) and printed.
- **Case 2 (Null Values):** User ID 2 has null email and age. The ?? operator displays 'Not Provided' and 'Not Specified'.
- **Case 3 (Not Found):** User ID 99 does not exist on server. The function throws an exception, and try-catch catches it.
- **Case 4 (Invalid Input):** User ID -5 is invalid (≤ 0). It throws an exception, and try-catch handles it cleanly.

5. Quick Summary Table

- **String:** Cannot be null (e.g., String name = 'John';)
- **String?:** Can be null (e.g., String? email = null;)
- **a ?? b:** If 'a' is not null use 'a', otherwise use 'b'
- **Future<T>:** A placeholder for data of type T that takes time to fetch
- **async:** Placed after function name to enable await
- **await:** Waits for a Future to complete
- **try-catch:** Handles errors safely without crashing the app